Parameter Properties

TypeScript offers an elegant shortcut syntax for creating constructors, which significantly reduces boilerplate code. It simultaneously aids in defining and assigning properties within a class. Let's dive in and explore how this works.

Traditional Approach to Constructors: In a traditional constructor, we often define parameters and then explicitly assign those parameters to the given properties within the class.

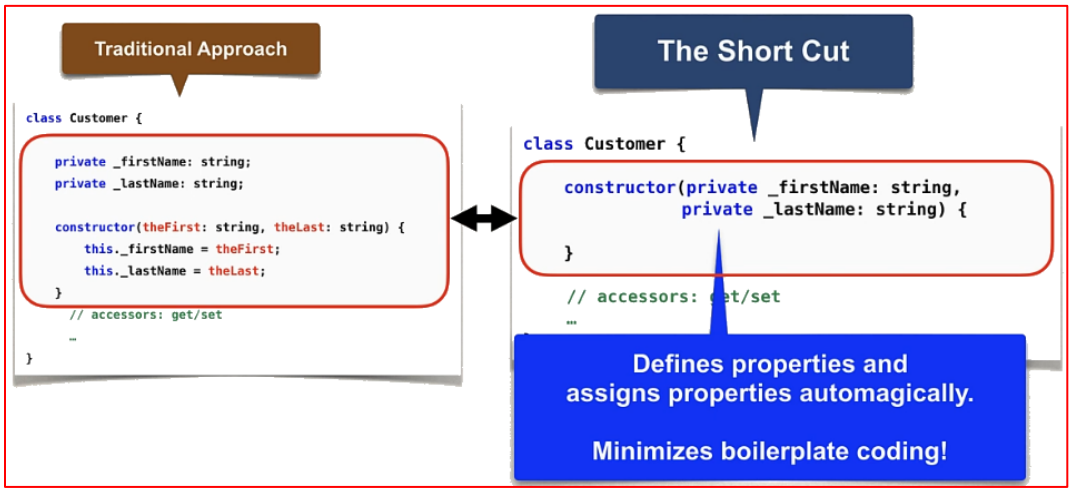


TypeScript's Shortcut: Parameter Properties: But here's where TypeScript shines! It offers an innovative shortcut that achieves the same functionality in a more succinct manner.

Imagine you're looking at two code snippets side by side. On the left, you see the traditional approach, and on the right, you see TypeScript's shortcut using parameter properties.

Notice something special in the shortcut code:

- The constructor still has two parameters.
- You'll spot the access modifiers, such as `private _firstName, private _lastName`.
- These access modifiers define the properties and assign them simultaneously, eliminating redundant code.



Comparing the Approaches: This new method, utilizing parameter properties, is at the same level of coding as the traditional approach. It's more concise and automates defining and assigning properties, making your coding life easier when setting up classes.

Seeing Parameter Properties in Action: Let's examine a practical example with a class named `Customer`, employing parameter properties. What's wonderful is that to the program using this class, everything operates identically, except the underlying constructor is more concise.

- **Creating an Instance**: You simply create a new instance by passing in the first name, last name, and access properties as usual.
- **The Difference**: When defining or creating the actual `Customer` class, the use of parameter properties saves you from some boilerplate coding, simplifying the process.

Let's Put It to the Test: Now, we're going to switch to our IDE and experiment with this coding technique.

1. **Preparation**: Start with some housekeeping work, like copying your previous project. Rename it to `04-parameter-properties`.
2. **Editing the Customer File**: Open the customer file in the `04-parameter-properties` folder. Remove the traditional coding for the constructor and use the parameter properties shortcut.
3. **Refactoring the Code**: Replace the traditional property declarations with parameter properties. This involves cutting certain portions of code and pasting them as parameters. TypeScript handles everything else automatically behind the scenes.
4. **Cleaning Up**: Get rid of unnecessary lines and internal assignments. Compare your new code with the old, and you'll notice how much cleaner and efficient it is using parameter properties.
5. **Compile and Run**: Open a terminal window, navigate to the directory `04-parameter-properties`, and execute a clean build using `tsc --build --clean` (followed by a `tsc --init` command). Then, compile the code as usual: `tsc` and run it with `node` command.

Your output remains the same because you're still constructing the object using parameter properties. The only alteration is that you've tidied up your coding by leveraging this wonderful TypeScript shortcut.

Summary: TypeScript's parameter properties offer an efficient and elegant way to define and initialize properties in a class constructor. It's a powerful tool that promotes concise, maintainable code. We encourage you to explore this feature further in your TypeScript development.